

Группа 212

Корепин Матвей

Эссе по первым шести занятиям курса OTUS «Разработчик C++»

Занятие 1. Вводное занятие

Темы занятия: организация курса, знакомство с системой контроля версий Git, работа с репозиторием на GitHub, понятие ветки и коммита, сравнение версий кода.

Система контроля версий хранит историю изменений исходного кода и позволяет узнать, кто и когда что изменил, сравнить любые версии файла и объединить изменения нескольких участников. **Репозиторий** — это хранилище проекта вместе с историей его изменений. **Ветка** — независимая линия разработки, которая не затрагивает основной код до момента слияния.

Я создал аккаунт и репозиторий на GitHub, создал в нём две ветки (test1 и test2), внёс в них разные изменения и сравнил их между собой. Наглядно увидел, как система контроля версий показывает построчные отличия между двумя версиями одного и того же файла. Итог занятия — не программа, а настроенный репозиторий, в котором я буду вести дальнейшие домашние задания.

Затруднение вызвала разница между локальным репозиторием на компьютере и удалённым репозиторием на GitHub — пришлось отдельно разобраться, как команды git push и git pull синхронизируют их между собой.

Занятие 2. Пишем свой Hello, World

Темы занятия: первая программа на языке C++, сборка проекта через CMake, разделение кода на заголовочный файл с объявлением функции и файл с её реализацией.

Заголовочный файл содержит только объявление функции — имя, параметры и тип возвращаемого значения, без самого кода. **Файл реализации** содержит тело функции. Такое разделение позволяет подключать функцию в других файлах через директиву `include`, не видя её внутреннего устройства. **CMake** — инструмент автоматизации сборки, который по текстовому описанию `CMakeLists.txt` собирает проект.

Я собрал исходную программу `hello_world.cpp` из материалов занятия, а затем выполнил задание: вынес реализацию функции `todo_move_me_out` из `hello_world.cpp` в отдельную пару файлов `move_me_out.h` и `move_me_out.cpp`, не меняя код функции `main`, и добавил новые файлы в `CMakeLists.txt`. Программа состоит из четырёх файлов: `hello_world.cpp` с функцией `main`, пары `example.h/example.cpp` с вспомогательной функцией из материалов занятия и пары `move_me_out.h/move_me_out.cpp` с вынесенной мной функцией. Проверил, что программа по-прежнему собирается и работает так же, как раньше.

Затруднение: сначала было неясно, зачем разделять объявление и реализацию функции, если можно писать всё в одном файле. Стало понятнее, когда на следующем занятии я увидел, что именно на этом разделении строится идея библиотек.

Занятие 3. Базовый синтаксис языка C++

Темы занятия: ввод и вывод через `cin` и `cout`, пространства имён, диапазоны целочисленных и вещественных типов данных, условные операторы `if` и `switch`, циклы `while`, `do-while` и `for`, файловый ввод-вывод, приведение типов.

Пространство имён группирует имена и позволяет избегать конфликтов между ними — вся стандартная библиотека C++ находится в пространстве имён `std`. **Диапазон типа** — это множество значений, которые может хранить переменная данного типа; выход за границы диапазона (переполнение) даёт непредсказуемый результат.

Отдельного задания с новой программой к этому занятию не было. Я разобрал и запустил примеры из материалов занятия, среди них — пример с диапазонами целочисленных типов, который экспериментально показывает эффект переполнения, и примеры с циклами и условными операторами. Каждый пример я собрал через CMake и проверил его вывод, чтобы понять назначение каждой конструкции языка.

Затруднение вызвала разница между циклами `while` и `do-while` — стало понятно только после того, как я подставил заведомо ложное условие в обе версии цикла и сравнил результат: цикл `do-while` в любом случае выполняет тело хотя бы один раз.

Занятие 4. Такие разные функции

Темы занятия: способы передачи параметров в функции (по значению, по ссылке, по константной ссылке, по указателю), параметры функции `main` и аргументы командной строки, перегрузка функций, значения параметров по умолчанию, рекурсия и стек вызовов.

Передача параметра по значению создаёт копию аргумента, и изменения этой копии не затрагивают исходную переменную. **Передача по ссылке** даёт функции прямой доступ к исходной переменной, что нужно для изменяемых (выходных) параметров, например при обмене значений двух переменных. **Передача по константной ссылке** не копирует объект, но и не позволяет функции его изменить — используется для больших объектов, которые не нужно менять. **Перегрузка функций** — это возможность объявить несколько функций с одинаковым именем, но разным набором параметров. **Рекурсия** — это вызов функции самой собой; без чёткого условия остановки рекурсия приводит к переполнению стека вызовов.

Отдельного задания к этому занятию тоже не было — я разобрал примеры из материалов: передачу параметров по значению на примере алгоритма Евклида, передачу по ссылке на примере функции обмена

значений, передачу по константной ссылке на примере работы со строкой, разбор аргументов командной строки, перегрузку функции для векторов разной размерности, рекурсивное решение задачи о Ханойских башнях и наивную рекурсию для чисел Фибоначчи, которая наглядно показывает, что рекурсия не всегда эффективна: время её работы растёт экспоненциально, потому что одни и те же значения пересчитываются заново много раз.

Затруднение вызвало большое количество способов передачи параметров. Разобрался, только когда сопоставил каждый способ с конкретной задачей: значение — когда данные не должны меняться, ссылка — когда функция обязана изменить переданную переменную, константная ссылка — когда объект дорогой для копирования, но менять его нельзя.

Занятие 5. Модульность. Линковка

Темы занятия: файловый ввод-вывод, этапы сборки программы (препроцессинг, компиляция, ассемблирование, линковка), понятие единицы трансляции, статические и динамические библиотеки.

Превращение исходного текста программы в исполняемый файл проходит несколько этапов. **Препроцессинг** заменяет директивы вроде `include` и `define` чисто текстовой подстановкой. **Компилятор** проверяет код и превращает его в ассемблерный код. **Ассемблер** превращает его в машинный код, но пока без связей между разными файлами программы. **Линковщик** на основе таблиц символов каждого объектного файла связывает все части программы в единый исполняемый файл. **Единица трансляции** — это один `src`-файл вместе со всем содержимым, которое в него подставили директивы `include`. **Статическая библиотека** копируется в каждый использующий её исполняемый файл, а **динамическая библиотека** хранится отдельно и может использоваться несколькими программами одновременно.

Я разобрал примеры файлового ввода-вывода, перегрузки функций, статической и динамической библиотеки из материалов занятия. В качестве домашнего задания написал программу «Угадай число» с сохранением результатов в таблицу рекордов, выполнив также все пять дополнительных заданий: ограничение диапазона через параметр `-max`, вывод только таблицы рекордов через параметр `-table`, выбор уровня сложности через параметр `-level`, хранение по каждому игроку только его минимального числа попыток и автоматическую перезапись результата, если новый оказался лучше. Программа разделена на три файла: один отвечает за разбор аргументов командной строки, второй — за чтение и запись таблицы рекордов, третий связывает игру воедино, используя подходы из заготовок, приложенных к заданию.

Затруднение: при первом тестировании программа зависала, если ввод заканчивался раньше, чем игрок угадывал число. Пришлось добавить проверку на неудачное чтение из потока ввода и корректно завершать программу в этом случае.

Занятие 6. Подробнее о CMake

Темы занятия: проблема переносимости сборки программы между разными окружениями, CMake как кроссплатформенная система сборки, понятие генератора CMake, структура файла `CMakeLists.txt` для проектов разной сложности.

Проблема «у меня работает» возникает, когда программа собирается и запускается у одного разработчика, но ломается на другом компьютере из-за разных версий компилятора, библиотек, операционной системы или архитектуры процессора. CMake решает эту проблему тем, что сборка проекта описывается один раз в терминах, не зависящих от конкретного инструмента сборки, а затем генератор CMake создаёт из этого описания нужные сборочные файлы под конкретную платформу.

Я разобрал последовательность примеров из материалов занятия — от простейшего проекта с одним исполняемым файлом до более сложных проектов с отдельными подключаемыми библиотеками и генерацией заголовочного файла с версией проекта. Это напрямую опирается на понятия предыдущего занятия — единицу трансляции и библиотеки, только теперь показано, как описать их декларативно, а не собирать вручную.

Затруднение вызвал смысл файла `config.h.in` и генерация из него заголовочного файла во время сборки — казалось, что проще сразу написать обычный заголовочный файл. Стало понятно, когда я увидел, что через этот механизм в код автоматически подставляются значения из `CMakeLists.txt`, например номер версии проекта, без ручного редактирования кода.